贪心算法

分发饼干

455. 分发饼干

已解答

简单 相关标签 相关企业 A文

假设你是一位很棒的家长，想要给你的孩子们一些小饼干。但是，每个孩子最多只能给一块饼干。

对每个孩子 `i`，都有一个胃口值 `g[i]`，这是能让孩子们满足胃口的饼干的最小尺寸；并且每块饼干 `j`，都有一个尺寸 `s[j]` 。如果 `s[j] >= g[i]`，我们可以将这个饼干 `j` 分配给孩子 `i` ，这个孩子会得到满足。你的目标是满足尽可能多的孩子，并输出这个最大数值。

示例 1:

> **输入：** g = [1,2,3], s = [1,1]
> **输出：** 1
> **解释：**
> 你有三个孩子和两块小饼干，3 个孩子的胃口值分别是：1,2,3。
> 虽然你有两块小饼干，由于他们的尺寸都是 1，你只能让胃口值是 1 的孩子满足。
> 所以你应该输出 1。

示例 2:

> **输入：** g = [1,2], s = [1,2,3]
> **输出：** 2
> **解释：**
> 你有两个孩子和三块小饼干，2 个孩子的胃口值分别是 1,2。
> 你拥有的饼干数量和尺寸都足以让所有孩子满足。
> 所以你应该输出 2。

- 可以看出来，是先遍历的胃口，在遍历的饼干，那么可不可以 先遍历 饼干，在遍历胃口呢?
- 其实是不可以的。
- 外面的 for 是里的下标 i 是**固定移动**的，而 if 里面的下标 index 是**符合条件才移动**的

```
int findContentChildren(vector<int>& g, vector<int>& s) {
        //必须先排序
        sort(g.begin(), g.end());
        sort(s.begin(), s.end());
        int index = s.size() - 1; // 饼干数组的下标
        int result = 0;
        //大饼干先满足大胃口
        for (int i = g.size() - 1; i >= 0; i--) { // 遍历胃口
            //记得index >= 0 , 使数组不越界
            if (index >= 0 && s[index] >= g[i]) { // 遍历饼干
                result++;
```

```
                index--;
            }
        }
        /*也可以从小饼干先满足小胃口
        for (int i = g.size() - 1; i >= 0; i--) { // 遍历胃口
            if (index >= 0 && s[index] >= g[i]) { // 遍历饼干
                result++;
                index--;
            }
        }*/
        return result;
    }
```

摆动序列

376. 摆动序列

中等 相关标签 相关企业 A文

如果连续数字之间的差严格地在正数和负数之间交替，则数字序列称为 **摆动序列** 。第一个差（如果存在的话）可能是正数或负数。仅有一个元素或者含两个不等元素的序列也视作摆动序列。

- 例如， `[1, 7, 4, 9, 2, 5]` 是一个 **摆动序列** ，因为差值 `(6, -3, 5, -7, 3)` 是正负交替出现的。
- 相反， `[1, 4, 7, 2, 5]` 和 `[1, 7, 4, 5, 5]` 不是摆动序列，第一个序列是因为它的前两个差值都是正数，第二个序列是因为它的最后一个差值为零。

子序列 可以通过从原始序列中删除一些（也可以不删除）元素来获得，剩下的元素保持其原始顺序。

给你一个整数数组 `nums` ，返回 `nums` 中作为 **摆动序列** 的 **最长子序列的长度** 。

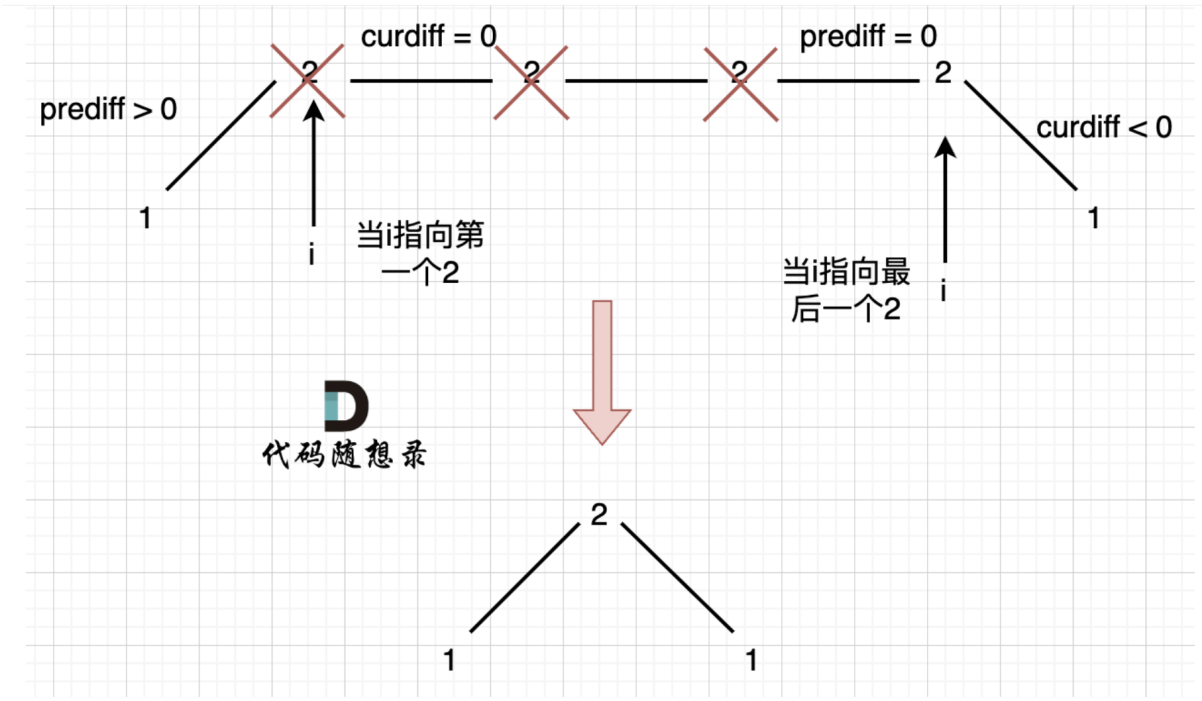
示例 1：

> **输入：** `nums = [1,7,4,9,2,5]`
> **输出：** `6`
> **解释：** 整个序列均为摆动序列，各元素之间的差值为 `(6, -3, 5, -7, 3)` 。

示例 2：

- 有三种情况：

- 上下坡中有平坡



在图中，当 i 指向第一个 2 的时候， `prediff > 0 && curdiff = 0` ，当 i 指向最后一个 2 的时候 `prediff = 0 && curdiff < 0` 。

如果我们采用，删左面三个 2 的规则，那么 当 `prediff = 0 && curdiff < 0` 也要记录一个峰值，因为他是把之前相同的元素都删掉留下的峰值。

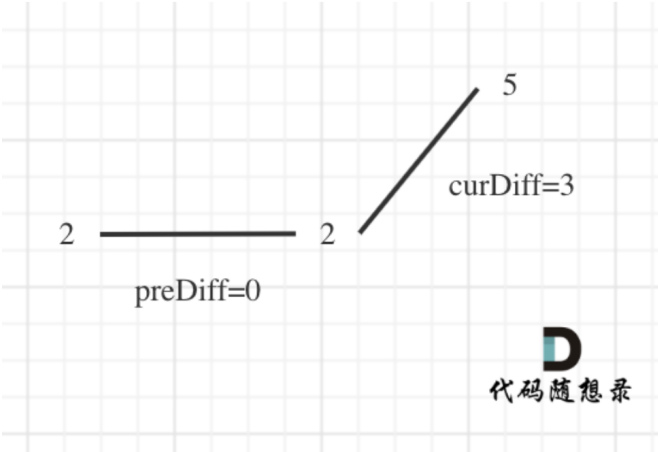
所以我们记录峰值的条件应该是： `(preDiff <= 0 && curDiff > 0) || (preDiff >= 0 && curDiff < 0)` ，为什么这里允许 prediff == 0 ，就是为了 上面我说的这种情况。

- 数组首尾两端

可以假设，数组最前面还有一个数字，那这个数字应该是什么呢？

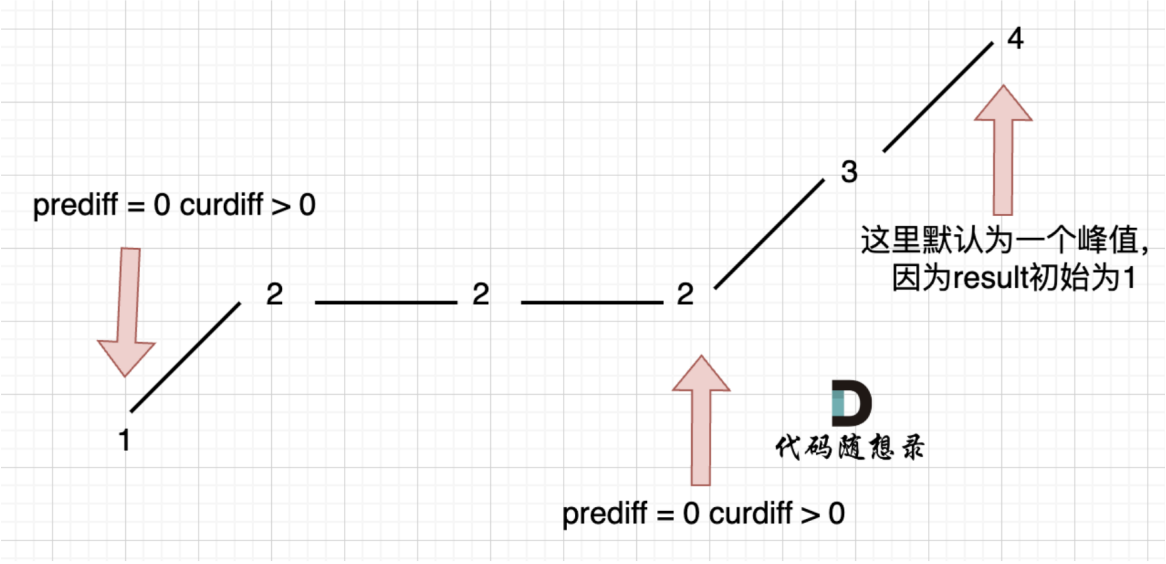
之前我们在 讨论 情况一：相同数字连续 的时候， prediff = 0 ，curdiff < 0 或者 >0 也记为波谷。

那么为了规则统一，针对序列[2,5]，可以假设为[2,2,5]，这样它就有坡度了即 preDiff = 0，如图：



针对以上情形，result 初始为 1（默认最右面有一个峰值），此时 curDiff > 0 && preDiff <= 0，那么 result++（计算了左面的峰值），最后得到的 result 就是 2（峰值个数为 2 即摆动序列长度为 2）

- 单调坡中有平坡



图中，我们可以看出，版本一的代码在三个地方记录峰值，但其实结果因为是 2，因为 单调中的平坡 不能算峰值（即摆动）。

之所以版本一会出问题，是因为我们实时更新了 prediff。

那么我们应该什么时候更新 prediff 呢？

我们只需要在 这个坡度 摆动变化的时候，更新 prediff 就行，这样 prediff 在 单调区间有平坡的时候 就不会发生变化，造成我们的误判。

```
int wiggleMaxLength(vector<int>& nums) {
        if (nums.size() <= 1) return nums.size();
        int curDiff = 0; // 当前一对差值
        int preDiff = 0; // 前一对差值
        int result = 1;  // 记录峰值个数，序列默认序列最右边有一个峰值
        for (int i = 0; i < nums.size() - 1; i++) {
            curDiff = nums[i + 1] - nums[i];
            // 出现峰值
            if ((preDiff <= 0 && curDiff > 0) || (preDiff >= 0 && curDiff < 0)) {
                result++;
                preDiff = curDiff; // 注意这里，只在摆动变化的时候更新prediff
            }
        }
        return result;
    }
```

最大子数组

53. 最大子数组和

中等 相关标签 相关企业 A文

给你一个整数数组 `nums` ，请你找出一个具有最大和的连续子数组（子数组最少包含一个元素），返回其最大和。

子数组是数组中的一个连续部分。

```
int maxSubArray(vector<int>& nums) {
        int result = INT32_MIN;
        int count = 0;
        for (int i = 0; i < nums.size(); i++) {
            count += nums[i];
            if (count > result) { // 取区间累计的最大值（相当于不断确定最大子序终止位置）
                result = count;
            }
            if (count <= 0) count = 0; // 相当于重置最大子序起始位置，因为遇到负数一定是拉低总和
        }
        return result;
    }
```

买卖股票的最大利润

122. 买卖股票的最佳时机 II

已解答

中等 相关标签 相关企业 A文

给你一个整数数组 `prices` ，其中 `prices[i]` 表示某支股票第 `i` 天的价格。

在每一天，你可以决定是否购买和/或出售股票。你在任何时候 **最多** 只能持有 **一股** 股票。然而，你可以在 **同一天** 多次买卖该股票，但要确保你持有的股票不超过一股。

返回 *你能获得的 **最大** 利润* 。

示例 1：

> **输入：** prices = [7,1,5,3,6,4]
> **输出：** 7
> **解释：** 在第 2 天（股票价格 = 1）的时候买入，在第 3 天（股票价格 = 5）的时候卖出，这笔交易所能获得利润 = 5 - 1 = 4。
> 随后，在第 4 天（股票价格 = 3）的时候买入，在第 5 天（股票价格 = 6）的时候卖出，这笔交易所能获得利润 = 6 - 3 = 3。
> 最大总利润为 4 + 3 = 7 。

示例 2：

> **输入：** prices = [1,2,3,4,5]
> **输出：** 4
> **解释：** 在第 1 天（股票价格 = 1）的时候买入，在第 5 天 （股票价格 = 5）的时候卖出，这笔交易所能获得利润 = 5 - 1 = 4。
> 最大总利润为 4 。

```
int maxProfit(vector<int>& prices) {
        int result = 0;
        for(int i = 1; i < prices.size(); i ++) {
            result += max(prices[i] - prices[i - 1], 0);
        }
        return result;
    }
```

跳跃游戏I

55. 跳跃游戏

已解答

中等 相关标签 相关企业 A文

给你一个非负整数数组 `nums` ，你最初位于数组的 **第一个下标** 。数组中的每个元素代表你在该位置可以跳跃的最大长度。

判断你是否能够到达最后一个下标，如果可以，返回 `true` ；否则，返回 `false` 。

示例 1：

> **输入：** nums = [2,3,1,1,4]
> **输出：** true
> **解释：** 可以先跳 1 步，从下标 0 到达下标 1，然后再从下标 1 跳 3 步到达最后一个下标。

示例 2：

> **输入：** nums = [3,2,1,0,4]
> **输出：** false
> **解释：** 无论怎样，总会到达下标为 3 的位置。但该下标的最大跳跃长度是 0 ， 所以永远不可能到达最后一个下标。

```
bool canJump(vector<int>& nums) {
        int cover = 0;
        if (nums.size() == 1) return true; // 只有一个元素，就是能达到
        for (int i = 0; i <= cover; i++) { // 注意这里是小于等于cover
            cover = max(i + nums[i], cover);
            if (cover >= nums.size() - 1) return true; // 说明可以覆盖到终点了
        }
        return false;
    }
```

跳跃游戏II

45. 跳跃游戏 II

已解答

中等 相关标签 相关企业 A文

给定一个长度为 `n` 的 **0 索引**整数数组 `nums`。初始位置在下标 0。

每个元素 `nums[i]` 表示从索引 `i` 向后跳转的最大长度。换句话说，如果你在索引 `i` 处，你可以跳转到任意 `(i + j)` 处：

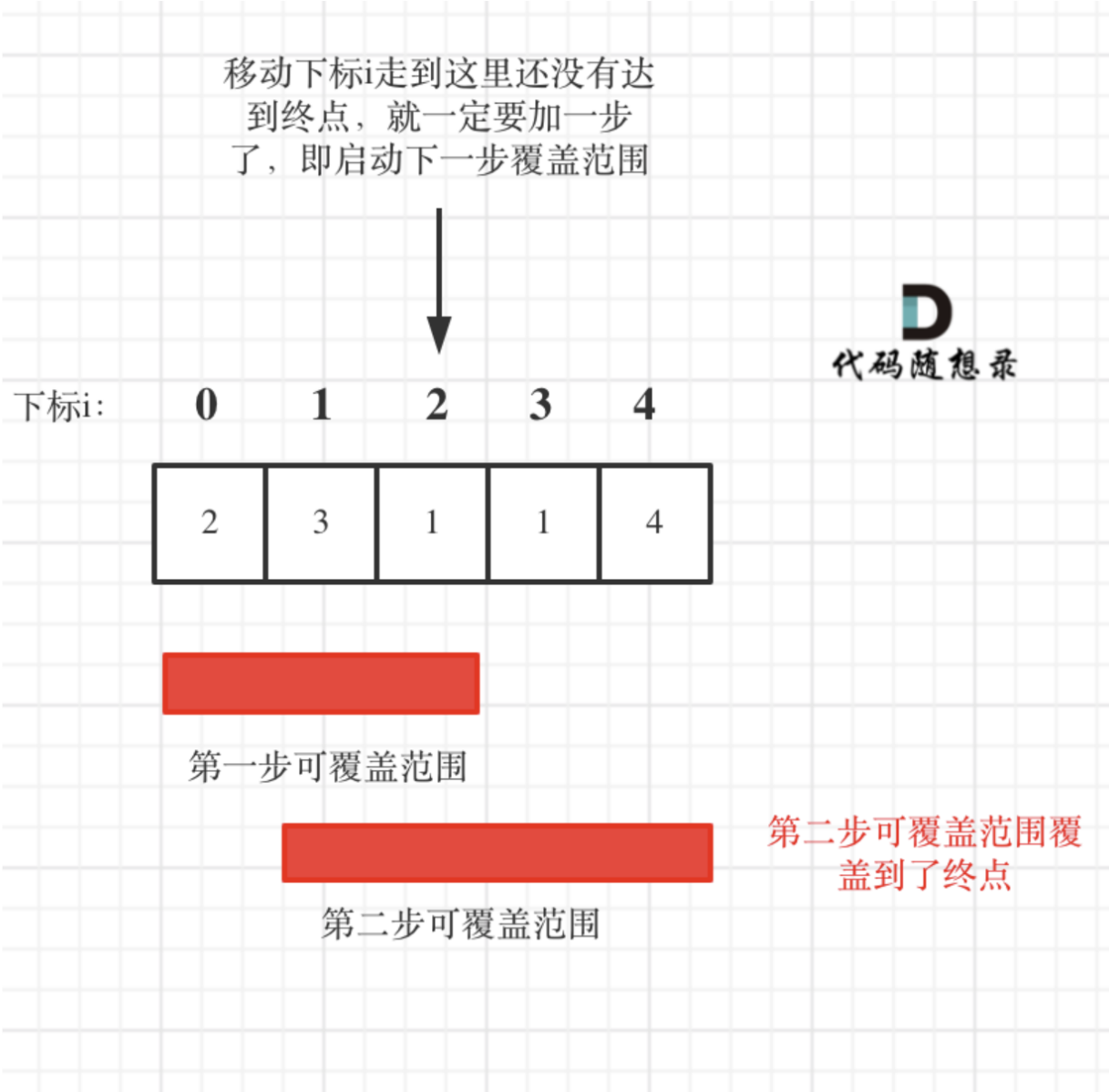
- `0 <= j <= nums[i]` 且
- `i + j < n`

返回到达 `n - 1` 的最小跳跃次数。测试用例保证可以到达 `n - 1`。

示例 1:

> **输入：** `nums = [2,3,1,1,4]`
> **输出：** `2`
> **解释：** 跳到最后一个位置的最小跳跃数是 `2`。
> 从下标为 `0` 跳到下标为 `1` 的位置，跳 `1` 步，然后跳 `3` 步到达数组的最后一个位置。

- 要从覆盖范围出发，不管怎么跳，覆盖范围内一定是可以跳到的，以最小的步数增加覆盖范围，覆盖范围一旦覆盖了终点，得到的就是最少步数！
- 这里需要统计两个覆盖范围，当前这一步的最大覆盖和下一步最大覆盖。
- 如果移动下标达到了当前这一步的最大覆盖最远距离了，还没有到终点的话，那么就必须再走一步来增加覆盖范围，直到覆盖范围覆盖了终点。



- **图中覆盖范围的意义在于，只要红色的区域，最多两步一定可以到！（不用管具体怎么跳，反正一定可以跳到）**
- 特殊情况：
  - 如果当前覆盖最远距离下标不是是集合终点，步数就加一，还需要继续走
  - 如果当前覆盖最远距离下标就是是集合终点，步数不用加一，因为不能再往后走了

```
int jump(vector<int>& nums) {
        if (nums.size() == 1) return 0;
        int curDistance = 0;    // 当前覆盖最远距离下标
        int ans = 0;            // 记录走的最大步数
        int nextDistance = 0;   // 下一步覆盖最远距离下标
        for (int i = 0; i < nums.size(); i++) {
            nextDistance = max(nums[i] + i, nextDistance);  // 更新下一步覆盖最远距
离下标
            if (i == curDistance) {        // 遇到当前覆盖最远距离下标
                ans++;                                  // 需要走下一步
                curDistance = nextDistance;    // 更新当前覆盖最远距离下标（相当于加
油了）
                if (nextDistance >= nums.size() - 1) break;
                // 当前覆盖最远距到达集合终点，不用做ans++操作了，直接结束
```

```
            }
        }
        return ans;
    }
```

K次取反后最大化的数组和

1005. K 次取反后最大化的数组和

已解答

简单 相关标签 相关企业 A文

给你一个整数数组 `nums` 和一个整数 `k` ，按以下方法修改该数组：

- 选择某个下标 `i` 并将 `nums[i]` 替换为 `-nums[i]` 。

重复这个过程恰好 `k` 次。可以多次选择同一个下标 `i` 。

以这种方式修改数组后，返回数组 **可能的最大和** 。

示例 1：

> **输入：** nums = [4,2,3], k = 1
> **输出：** 5
> **解释：** 选择下标 1 ，nums 变为 [4,-2,3] 。

示例 2：

> **输入：** nums = [3,-1,0,2], k = 3
> **输出：** 6
> **解释：** 选择下标 (1, 2, 2) ，nums 变为 [3,1,0,2] 。

```
class Solution {
static bool cmp(int a, int b) {
    return abs(a) > abs(b);
}
public:
    int largestSumAfterKNegations(vector<int>& A, int K) {
        sort(A.begin(), A.end(), cmp);       // 第一步
        for (int i = 0; i < A.size(); i++) { // 第二步
            if (A[i] < 0 && K > 0) {
                A[i] *= -1;
                K--;
            }
        }
        if (K % 2 == 1) A[A.size() - 1] *= -1; // 第三步
        int result = 0;
        for (int a : A) result += a;        // 第四步
```

```
        return result;
    }
};
```

加油站

134. 加油站

已解答

中等 相关标签 相关企业 A文

在一条环路上有 `n` 个加油站，其中第 `i` 个加油站有汽油 `gas[i]` 升。

你有一辆油箱容量无限的的汽车，从第 `i` 个加油站开往第 `i+1` 个加油站需要消耗汽油 `cost[i]` 升。你从其中的一个加油站出发，开始时油箱为空。

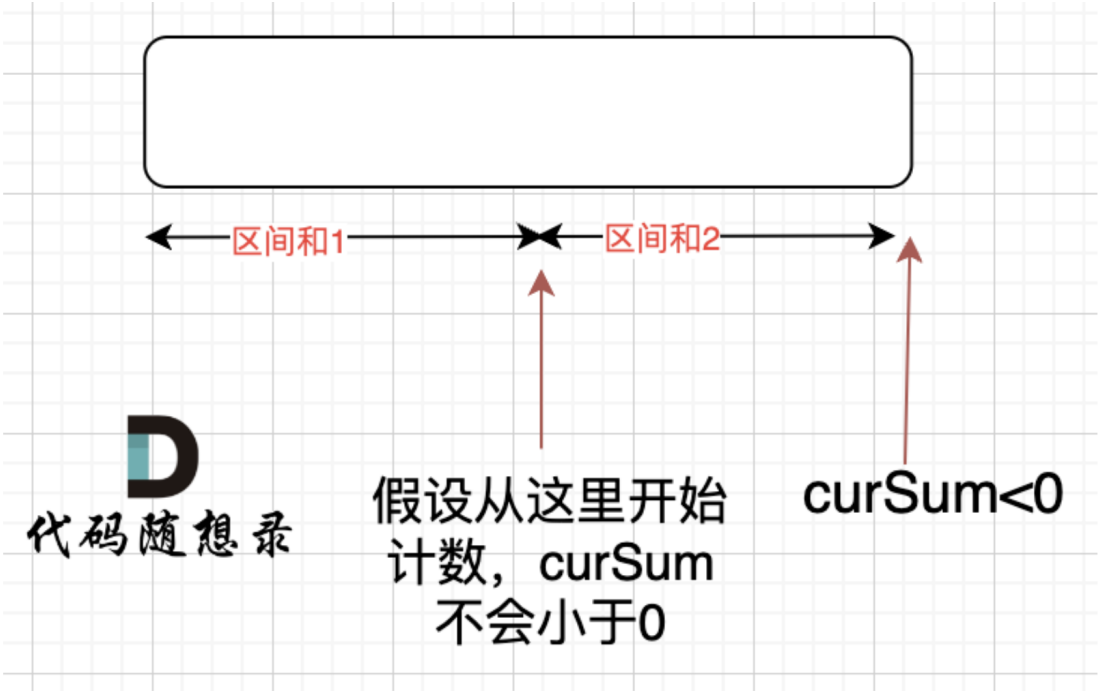
给定两个整数数组 `gas` 和 `cost` ，如果你可以按顺序绕环路行驶一周，则返回出发时加油站的编号，否则返回 `-1` 。如果存在解，则 **保证** 它是 **唯一** 的。

示例 1:

> **输入：** gas = [1,2,3,4,5], cost = [3,4,5,1,2]
> **输出：** 3
> **解释：**
> 从 3 号加油站(索引为 3 处)出发，可获得 4 升汽油。此时油箱有 = 0 + 4 = 4 升汽油
> 开往 4 号加油站，此时油箱有 4 - 1 + 5 = 8 升汽油
> 开往 0 号加油站，此时油箱有 8 - 2 + 1 = 7 升汽油
> 开往 1 号加油站，此时油箱有 7 - 3 + 2 = 6 升汽油
> 开往 2 号加油站，此时油箱有 6 - 4 + 3 = 5 升汽油
> 开往 3 号加油站，你需要消耗 5 升汽油，正好足够你返回到 3 号加油站。
> 因此，3 可为起始索引。

示例 2:

> **输入：** gas = [2,3,4], cost = [3,4,3]
> **输出：** -1



如果 curSum<0 说明 区间和1 + 区间和2 < 0， 那么 假设从上图中的位置开始计数curSum不会小于0的话，就是 区间和2>0。

区间和1 + 区间和2 < 0 同时 区间和2>0，只能说明区间和1 < 0， 那么就会从假设的箭头初就开始从新选择起始位置了。

那么局部最优：当前累加rest[i]的和curSum一旦小于0，起始位置至少要是i+1，因为从i之前开始一定不行。全局最优：找到可以跑一圈的起始位置。

这样的话这个中间点就也是重新标记的起始位置

```
int canCompleteCircuit(vector<int>& gas, vector<int>& cost) {
    int curSum = 0;
    int totalSum = 0;
    int start = 0;
    for (int i = 0; i < gas.size(); i++) {
        curSum += gas[i] - cost[i];
        totalSum += gas[i] - cost[i];
        if (curSum < 0) {   // 当前累加rest[i]和 curSum一旦小于0
            start = i + 1;  // 起始位置更新为i+1
            curSum = 0;     // curSum从0开始
        }
    }
    if (totalSum < 0) return -1; // 说明怎么走都不可能跑一圈了
    return start;
}
```

分发糖果

135. 分发糖果

已解答

困难 相关标签 相关企业 A文

`n` 个孩子站成一排。给你一个整数数组 `ratings` 表示每个孩子的评分。

你需要按照以下要求，给这些孩子分发糖果：

- 每个孩子至少分配到 `1` 个糖果。
- 相邻两个孩子中，评分更高的那个会获得更多的糖果。

请你给每个孩子分发糖果，计算并返回需要准备的 **最少糖果数目** 。

示例 1：

> **输入：** ratings = [1,0,2]
> **输出：** 5
> **解释：** 你可以分别给第一个、第二个、第三个孩子分发 2、1、2 颗糖果。

示例 2：

> **输入：** ratings = [1,2,2]
> **输出：** 4
> **解释：** 你可以分别给第一个、第二个、第三个孩子分发 1、2、1 颗糖果。
> 第三个孩子只得到 1 颗糖果，这满足题面中的两个条件。

- 一次是从左到右遍历，只比较右边孩子评分比左边大的情况
- 一次是从右到左遍历，只比较左边孩子评分比右边大的情况

```
int candy(vector<int>& ratings) {
    vector<int> candies(ratings.size(), 1);

    for(int i = 1; i < ratings.size(); i ++) {
        if(ratings[i] > ratings[i - 1]){
            candies[i] = candies[i - 1] + 1;
        }
    }

    for(int i = ratings.size() - 1; i > 0; i --) {
        if(ratings[i - 1] > ratings[i]) {
            candies[i - 1] = max(candies[i] + 1, candies[i -  1]);
        }
    }

    int sum = 0;
    for(int i = 0; i < candies.size(); i ++) {
        sum += candies[i];
    }
```

```
    return sum;
}
```

柠檬水找零

860. 柠檬水找零

已解答

简单 相关标签 相关企业 A文

在柠檬水摊上，每一杯柠檬水的售价为 `5` 美元。顾客排队购买你的产品，（按账单 `bills` 支付的顺序）一次购买一杯。

每位顾客只买一杯柠檬水，然后向你付 `5` 美元、`10` 美元或 `20` 美元。你必须给每个顾客正确找零，也就是说净交易是每位顾客向你支付 `5` 美元。

注意，一开始你手头没有任何零钱。

给你一个整数数组 `bills` ，其中 `bills[i]` 是第 `i` 位顾客付的账。如果你能给每位顾客正确找零，返回 `true` ，否则返回 `false` 。

示例 1：

> **输入：** bills = [5,5,5,10,20]
> **输出：** true
> **解释：**
> 前 3 位顾客那里，我们按顺序收取 3 张 5 美元的钞票。
> 第 4 位顾客那里，我们收取一张 10 美元的钞票，并返还 5 美元。
> 第 5 位顾客那里，我们找还一张 10 美元的钞票和一张 5 美元的钞票。
> 由于所有客户都得到了正确的找零，所以我们输出 true。

```
bool lemonadeChange(vector<int>& bills) {
    int five = 0, ten = 0, twenty = 0;
    for (int bill : bills) {
        // 情况一
        if (bill == 5) five++;
        // 情况二
        if (bill == 10) {
            if (five <= 0) return false;
            ten++;
            five--;
        }
        // 情况三
        if (bill == 20) {
            // 优先消耗10美元，因为5美元的找零用处更大，能多留着就多留着
            if (five > 0 && ten > 0) {
                five--;
```

```
                ten--;
                twenty++; // 其实这行代码可以删了，因为记录20已经没有意义了，不会用20来找零
            } else if (five >= 3) {
                five -= 3;
                twenty++; // 同理，这行代码也可以删了
            } else return false;
        }
    }
    return true;
}
```

根据身高重建队列

406. 根据身高重建队列

已解答

中等 相关标签 相关企业 提示 A文

假设有打乱顺序的一群人站成一个队列，数组 `people` 表示队列中一些人的属性（不一定按顺序）。每个 `people[i] = [$h_i$, $k_i$]` 表示第 `i` 个人的身高为 `$h_i$` ，前面 **正好** 有 `$k_i$` 个身高大于或等于 `$h_i$` 的人。

请你重新构造并返回输入数组 `people` 所表示的队列。返回的队列应该格式化为数组 `queue` ，其中 `queue[j] = [$h_j$, $k_j$]` 是队列中第 `j` 个人的属性（`queue[0]` 是排在队列前面的人）。

示例 1：

> **输入：** people = [[7,0],[4,4],[7,1],[5,0],[6,1],[5,2]]
> **输出：** [[5,0],[7,0],[5,2],[6,1],[4,4],[7,1]]
> **解释：**
> 编号为 0 的人身高为 5 ，没有身高更高或者相同的人排在他前面。
> 编号为 1 的人身高为 7 ，没有身高更高或者相同的人排在他前面。
> 编号为 2 的人身高为 5 ，有 2 个身高更高或者相同的人排在他前面，即编号为 0 和 1 的人。
> 编号为 3 的人身高为 6 ，有 1 个身高更高或者相同的人排在他前面，即编号为 1 的人。
> 编号为 4 的人身高为 4 ，有 4 个身高更高或者相同的人排在他前面，即编号为 0、1、2、3 的人。
> 编号为 5 的人身高为 7 ，有 1 个身高更高或者相同的人排在他前面，即编号为 1 的人。
> 因此 [[5,0],[7,0],[5,2],[6,1],[4,4],[7,1]] 是重新构造后的队列。

- 局部最优：优先按**身高高的people的k**来插入。插入操作过后的people满足队列属性
- 全局最优：最后都做完插入操作，整个队列满足题目队列属性

排序完的people： [[7,0], [7,1], [6,1], [5,0], [5,2], [4,4]]

插入的过程：

- 插入[7,0]：[[7,0]]
- 插入[7,1]：[[7,0],[7,1]]
- 插入[6,1]：[[7,0],[6,1],[7,1]]
- 插入[5,0]：[[5,0],[7,0],[6,1],[7,1]]
- 插入[5,2]：[[5,0],[7,0],[5,2],[6,1],[7,1]]
- 插入[4,4]：[[5,0],[7,0],[5,2],[6,1],[4,4],[7,1]]

```
class Solution {
public:
    // 身高从大到小排（身高相同k小的站前面）
    static bool cmp(const vector<int>& a, const vector<int>& b) {
        if (a[0] == b[0]) return a[1] < b[1];
        return a[0] > b[0];
    }
    vector<vector<int>> reconstructQueue(vector<vector<int>>& people) {
        sort (people.begin(), people.end(), cmp);
        list<vector<int>> que; // list底层是链表实现，插入效率比vector高的多
        for (int i = 0; i < people.size(); i++) {
            int position = people[i][1]; // 插入到下标为position的位置
            std::list<vector<int>>::iterator it = que.begin();
            while (position--) { // 寻找在插入位置
                it++;
            }
            que.insert(it, people[i]);
        }
        return vector<vector<int>>(que.begin(), que.end());
    }
};
```

- 用list效率比vector高很多，后面要变回vector
- 其技巧是**确定一边然后贪心另一边**，两边一起考虑，就会顾此失彼。

用最少的数量箭引爆气球

452. 用最少数量的箭引爆气球

已解答

中等 相关标签 相关企业 A文

有一些球形气球贴在一堵用 XY 平面表示的墙面上。墙面上的气球记录在整数数组 `points` ，其中 `points[i] = [xstart, xend]` 表示水平直径在 `xstart` 和 `xend` 之间的气球。你不知道气球的确切 y 坐标。

一支弓箭可以沿着 x 轴从不同点 **完全垂直** 地射出。在坐标 `x` 处射出一支箭，若有一个气球的直径的开始和结束坐标为 `xstart`，`xend`，且满足 `xstart ≤ x ≤ xend`，则该气球会被 **引爆** 。可以射出的弓箭的数量 **没有限制** 。 弓箭一旦被射出之后，可以无限地前进。

给你一个数组 `points` ，*返回引爆所有气球所必须射出的 **最小** 弓箭数* 。

示例 1：

> **输入：** points = [[10,16],[2,8],[1,6],[7,12]]
> **输出：** 2
> **解释：** 气球可以用2支箭来爆破：
> -在x = 6处射出箭，击破气球[2,8]和[1,6]。
> -在x = 11处发射箭，击破气球[10,16]和[7,12]。

示例 2：

> **输入：** points = [[1,2],[3,4],[5,6],[7,8]]
> **输出：** 4
> **解释：** 每个气球需要射出一支箭，总共需要4支箭。

```
class Solution {
private:
    static bool cmp(const vector<int>& a, const vector<int>& b) {
        return a[0] < b[0];
    }
public:
    int findMinArrowShots(vector<vector<int>>& points) {
        if (points.size() == 0) return 0;
        sort(points.begin(), points.end(), cmp);

        int result = 1; // points 不为空至少需要一支箭
        for (int i = 1; i < points.size(); i++) {
            if (points[i][0] > points[i - 1][1]) {  // 气球i和气球i-1不挨着，注意这
里不是>=
                result++; // 需要一支箭
            }
            else {  // 气球i和气球i-1挨着
                points[i][1] = min(points[i - 1][1], points[i][1]); // 更新重叠气球
最小右边界
            }
        }
```

```
        return result;
    }
};
```

划分字母区间

763. 划分字母区间

已解答

中等 相关标签 相关企业 提示

给你一个字符串 `s` 。我们要把这个字符串划分为尽可能多的片段，同一字母最多出现在一个片段中。例如，字符串 `"ababcc"` 能够被分为 `["abab", "cc"]`，但类似 `["aba", "bcc"]` 或 `["ab", "ab", "cc"]` 的划分是非法的。

注意，划分结果需要满足：将所有划分结果按顺序连接，得到的字符串仍然是 `s` 。

返回一个表示每个字符串片段的长度的列表。

示例 1：

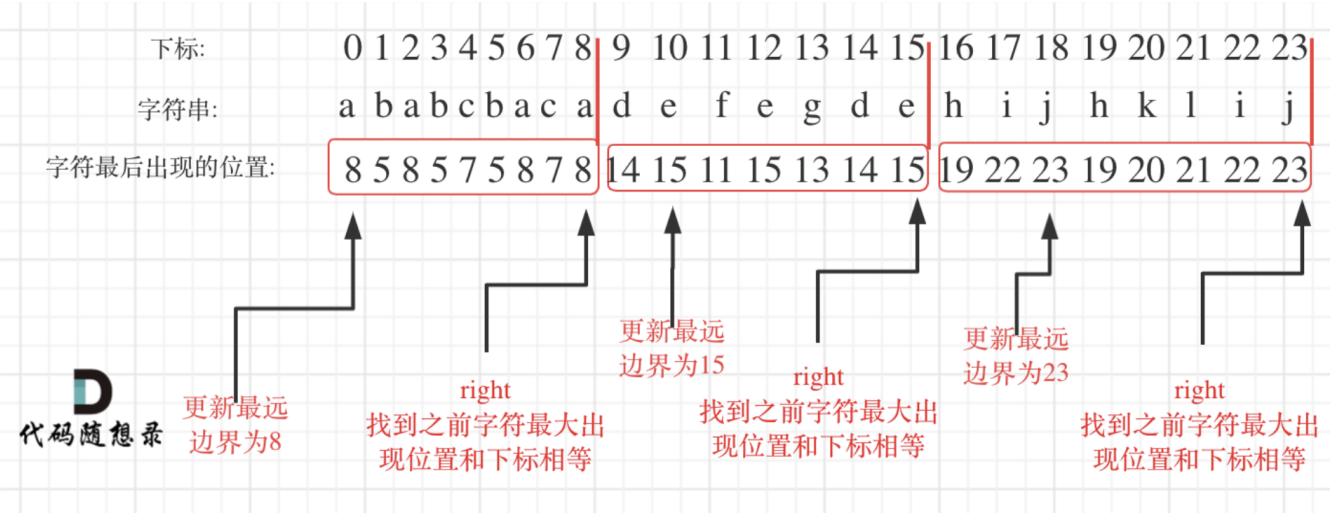
> **输入：** s = "ababcbacadefegdehijhklij"
> **输出：** [9,7,8]
> **解释：**
> 划分结果为 "ababcbaca"、"defegde"、"hijhklij" 。
> 每个字母最多出现在一个片段中。
> 像 "ababcbacadefegde", "hijhklij" 这样的划分是错误的，因为划分的片段数较少。

示例 2：

> **输入：** s = "eccbbbbdec"
> **输出：** [10]

- 统计每一个字符最后出现的位置
- 从头遍历字符，并更新字符的最远出现下标，如果找到字符最远出现位置下标和当前下标相等了，则找到了分割点

如图：




```
vector<int> partitionLabels(string S) {
    int hash[27] = {0}; // i为字符, hash[i]为字符出现的最后位置
    for (int i = 0; i < S.size(); i++) { // 统计每一个字符最后出现的位置
        hash[S[i] - 'a'] = i;
    }
    vector<int> result;
    int left = 0;
    int right = 0;
    for (int i = 0; i < S.size(); i++) {
        right = max(right, hash[S[i] - 'a']); // 找到字符出现的最远边界
        if (i == right) {
            result.push_back(right - left + 1);
            left = i + 1;
        }
    }
    return result;
}
```

合并区间

56. 合并区间

已解答

中等 相关标签 相关企业 A文

以数组 `intervals` 表示若干个区间的集合，其中单个区间为 `intervals[i] = [start_i, end_i]`。请你合并所有重叠的区间，并返回 *一个不重叠的区间数组，该数组需恰好覆盖输入中的所有区间*。

示例 1：

> **输入：** intervals = [[1,3],[2,6],[8,10],[15,18]]
> **输出：** [[1,6],[8,10],[15,18]]
> **解释：** 区间 [1,3] 和 [2,6] 重叠，将它们合并为 [1,6].

示例 2：

> **输入：** intervals = [[1,4],[4,5]]
> **输出：** [[1,5]]
> **解释：** 区间 [1,4] 和 [4,5] 可被视为重叠区间。

```
vector<vector<int>> merge(vector<vector<int>>& intervals) {
    vector<vector<int>> result;
    if (intervals.size() == 0) return result; // 区间集合为空直接返回
    // 排序的参数使用了lambda表达式
    sort(intervals.begin(), intervals.end(), [](const vector<int>& a, const
vector<int>& b){return a[0] < b[0];});

    // 第一个区间就可以放进结果集里，后面如果重叠，在result上直接合并
    result.push_back(intervals[0]);

    for (int i = 1; i < intervals.size(); i++) {
        if (result.back()[1] >= intervals[i][0]) { // 发现重叠区间
            // 合并区间，只更新右边界就好，因为result.back()的左边界一定是最小值，因为
我们按照左边界排序的
            result.back()[1] = max(result.back()[1], intervals[i][1]);
        } else {
            result.push_back(intervals[i]); // 区间不重叠
        }
    }
    return result;
}
```

单调递增的数字

738. 单调递增的数字

已解答

中等 相关标签 相关企业 提示 A文

当且仅当每个相邻位数上的数字 `x` 和 `y` 满足 `x <= y` 时，我们称这个整数是**单调递增**的。

给定一个整数 `n` ，返回 *小于或等于* `n` *的最大数字，且数字呈* ***单调递增*** 。

示例 1:

> **输入：** n = 10
> **输出：** 9

示例 2:

> **输入：** n = 1234
> **输出：** 1234

示例 3:

> **输入：** n = 332
> **输出：** 299

题目要求小于等于N的最大单调递增的整数，那么拿一个两位的数字来举例。

例如：98，一旦出现strNum[i - 1] > strNum[i]的情况（非单调递增），首先想让strNum[i - 1]--，然后strNum[i]给为9，这样这个整数就是89，即小于98的最大的单调递增整数。

这一点如果想清楚了，这道题就好办了。

此时是从前向后遍历还是从后向前遍历呢？

从前向后遍历的话，遇到strNum[i - 1] > strNum[i]的情况，让strNum[i - 1]减一，但此时如果strNum[i - 1]减一了，可能又小于strNum[i - 2]。

这么说有点抽象，举个例子，数字：332，从前向后遍历的话，那么就把变成了329，此时2又小于了第一位的3了，真正的结果应该是299。

那么从后向前遍历，就可以重复利用上次比较得出的结果了，从后向前遍历332的数值变化为：332 -> 329 -> 299

确定了遍历顺序之后，那么此时局部最优就可以推出全局，找不出反例，试试贪心。

```
int monotoneIncreasingDigits(int N) {
    string strNum = to_string(N);
    // flag用来标记赋值9从哪里开始
    // 设置为这个默认值，为了防止第二个for循环在flag没有被赋值的情况下执行
    int flag = strNum.size();
    for (int i = strNum.size() - 1; i > 0; i--) {
```

```
        if (strNum[i - 1] > strNum[i] ) {
            flag = i;
            strNum[i - 1]--;
        }
    }
    for (int i = flag; i < strNum.size(); i++) {
        strNum[i] = '9';
    }
    return stoi(strNum);
}
```

监控二叉树

968. 监控二叉树

已解答

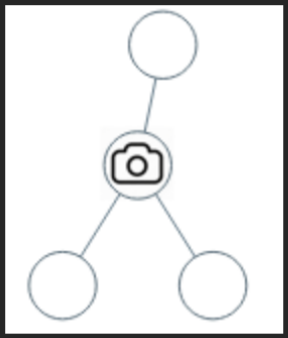
困难 相关标签 相关企业 A文

给定一个二叉树，我们在树的节点上安装摄像头。

节点上的每个摄影头都可以监视**其父对象、自身及其直接子对象。**

计算监控树的所有节点所需的最小摄像头数量。

示例 1：



> **输入：** [0,0,null,0,0]
> **输出：** 1
> **解释：** 如图所示，一台摄像头足以监控所有节点。

这道题目首先要想，如何放置，才能让摄像头最小的呢？

从题目中示例，其实可以得到启发，**我们发现题目示例中的摄像头都没有放在叶子节点上！**

这是很重要的一个线索，摄像头可以覆盖上中下三层，如果把摄像头放在叶子节点上，就浪费的一层的覆盖。

所以把摄像头放在叶子节点的父节点位置，才能充分利用摄像头的覆盖面积。

那么有同学可能问了，为什么不从头结点开始看起呢，为啥要从叶子节点看呢？

因为头结点放不放摄像头也就省下一个摄像头， 叶子节点放不放摄像头省下了的摄像头数量是指数阶别的。

所以我们要从下往上看，局部最优：让叶子节点的父节点安摄像头，所用摄像头最少，整体最优：全部摄像头数量所用最少！

局部最优推出全局最优，找不出反例，那么就按照贪心来！

此时，大体思路就是从低到上，先给叶子节点父节点放个摄像头，然后隔两个节点放一个摄像头，直至到二叉树头结点。

此时这道题目还有两个难点：

1. 二叉树的遍历
2. 如何隔两个节点放一个摄像头

- 该节点无覆盖
- 本节点有摄像头
- 本节点有覆盖

我们分别有三个数字来表示：

- 0：该节点无覆盖
- 1：本节点有摄像头
- 2：本节点有覆盖

大家应该找不出第四个节点的状态了。

一些同学可能会想有没有第四种状态：本节点无摄像头，其实无摄像头就是 无覆盖 或者 有覆盖的状态，所以一共还是三个状态。

因为在遍历树的过程中，就会遇到空节点，那么问题来了，空节点究竟是哪一种状态呢？ 空节点表示无覆盖？ 表示有摄像头？还是有覆盖呢？

回归本质，为了让摄像头数量最少，我们要尽量让叶子节点的父节点安装摄像头，这样才能摄像头的数量最少。

那么空节点不能是无覆盖的状态，这样叶子节点就要放摄像头了，空节点也不能是有摄像头的状态，这样叶子节点的父节点就没有必要放摄像头了，而是可以把摄像头放在叶子节点的爷爷节点上。

所以空节点的状态只能是有覆盖，这样就可以在叶子节点的父节点放摄像头了

```
class Solution {
private:
    int result;
    int traversal(TreeNode* cur) {

        // 空节点，该节点有覆盖
        if (cur == NULL) return 2;

        int left = traversal(cur->left);    // 左
        int right = traversal(cur->right);  // 右

        // 情况1
        // 左右节点都有覆盖
        if (left == 2 && right == 2) return 0;

        // 情况2
        // left == 0 && right == 0 左右节点无覆盖
        // left == 1 && right == 0 左节点有摄像头，右节点无覆盖
        // left == 0 && right == 1 左节点有无覆盖，右节点摄像头
        // left == 0 && right == 2 左节点无覆盖，右节点覆盖
        // left == 2 && right == 0 左节点覆盖，右节点无覆盖
        if (left == 0 || right == 0) {
            result++;
            return 1;
        }

        // 情况3
```

```
        // left == 1 && right == 2 左节点有摄像头，右节点有覆盖
        // left == 2 && right == 1 左节点有覆盖，右节点有摄像头
        // left == 1 && right == 1 左右节点都有摄像头
        // 其他情况前段代码均已覆盖
        if (left == 1 || right == 1) return 2;

        // 以上代码我没有使用else，主要是为了把各个分支条件展现出来，这样代码有助于读者
理解
        // 这个 return -1 逻辑不会走到这里。
        return -1;
    }

public:
    int minCameraCover(TreeNode* root) {
        result = 0;
        // 情况4
        if (traversal(root) == 0) { // root 无覆盖
            result++;
        }
        return result;
    }
};
```